Chapter 2: Intelligent Agents

1. Introduction to Agents and Environments

The chapter begins by introducing the concept of **agents** and their interaction with **environments**. An **agent** is anything that can perceive its environment through **sensors** and act upon that environment through **actuators**. For example:

- **Human agent**: Sensors are eyes, ears, etc.; actuators are hands, legs, etc.
- **Robotic agent**: Sensors are cameras, infrared sensors, etc.; actuators are motors, robot arms, etc.
- **Software agent**: Sensors are keystrokes, file contents, etc.; actuators are screen displays, file writes, etc.

The agent's **percept sequence** is the complete history of everything the agent has perceived. The agent's behavior is described by an **agent function**, which maps percept sequences to actions. This function can be represented as a table, but in practice, it's implemented as an **agent program** running on some physical system (like a computer or robot).

---

2. Rationality and Good Behavior

A **rational agent** is one that does the "right thing." But what does "right" mean? It depends on the **performance measure**, which evaluates the sequence of environment states resulting from the agent's actions. For example:

- In a **vacuum-cleaner world**, the performance measure could be the number of clean squares over time, with penalties for electricity consumption or noise.

Rationality depends on four factors:

1. The **performance measure**.
2. The agent's **prior knowledge** of the environment.
3. The **actions** the agent can perform.
4. The agent's **percept sequence** to date.

A rational agent selects actions that maximize its performance measure, given its percept sequence and prior knowledge.

---

3. Omniscience, Learning, and Autonomy

- **Omniscience**: An omniscient agent knows the actual outcome of its actions, but this is impossible in reality. Rationality is about maximizing **expected** performance, not actual performance.

- **Learning**: A rational agent should learn from its experiences to improve its performance over time.
- **Autonomy**: A rational agent should be autonomous, meaning it should rely on its own percepts and learning rather than solely on prior knowledge provided by its designer.

---

4. The Nature of Environments

The chapter introduces the **PEAS** framework for specifying the task environment:

- **Performance Measure**: What defines success?
- **Environment**: What is the agent operating in?
- **Actuators**: What actions can the agent take?
- **Sensors**: What information can the agent perceive?

For example, in an **automated taxi**:

- **Performance Measure**: Safe, fast, legal, comfortable trip, maximize profits.
- **Environment**: Roads, traffic, pedestrians, customers.
- **Actuators**: Steering, accelerator, brake, etc.
- **Sensors**: Cameras, GPS, speedometer, etc.

---

5. Properties of Task Environments

Task environments can be categorized along several dimensions:

1. **Fully Observable vs. Partially Observable**: Can the agent see the complete state of the environment?
2. **Single-Agent vs. Multi-Agent**: Is the agent operating alone or with other agents?
3. **Deterministic vs. Stochastic**: Is the next state of the environment completely determined by the current state and action, or is there randomness?
4. **Episodic vs. Sequential**: Is the agent's experience divided into independent episodes, or do current actions affect future decisions?
5. **Static vs. Dynamic**: Does the environment change while the agent is thinking?
6. **Discrete vs. Continuous**: Are the states, time, percepts, and actions discrete or continuous?
7. **Known vs. Unknown**: Does the agent know the rules of the environment?

More details :

These properties help us understand the nature of the environment in which an agent operates and how the agent should be designed to handle it.

---

1. Fully Observable vs. Partially Observable

- A **fully observable** environment means the agent's sensors provide complete information about the environment at all times. An example is chess, where the agent can see the entire board and all pieces. This eliminates the need for the agent to maintain an internal state.
- A **partially observable** environment means the agent has limited information. In poker, for example, the agent cannot see the opponents' cards. This requires maintaining an internal state to track hidden information.

2. Single-Agent vs. Multi-Agent

- A **single-agent** environment involves only one agent making decisions, such as a crossword puzzle solver.
- A **multi-agent** environment involves multiple agents that influence the outcome. Chess is an example of a competitive multi-agent environment, while a team of robots working together represents a cooperative multi-agent environment.

3. Deterministic vs. Stochastic

- A **deterministic** environment means the next state is completely determined by the current state and the agent's action. A vacuum-cleaner world with fixed dirt placement follows this model.
- A **stochastic** environment has randomness or uncertainty, such as taxi driving, where factors like traffic and weather introduce unpredictability.

4. Episodic vs. Sequential

- In an **episodic** environment, the agent's experience consists of independent episodes. Spam filtering is an example where each email is classified separately.
- In a **sequential** environment, current decisions influence future states, such as in chess, where each move affects the game's progress.

5. Static vs. Dynamic

- A **static** environment does not change while the agent is thinking, like a crossword puzzle.
- A **dynamic** environment changes while the agent is deciding, such as in taxi driving, where traffic conditions can shift unpredictably.

6. Discrete vs. Continuous

- A **discrete** environment has a finite number of distinct states, such as a chessboard with defined positions.
- A **continuous** environment has an infinite number of possible states, such as taxi driving, where speed and position vary continuously.

7. Known vs. Unknown

- A **known** environment has predefined rules, such as solitaire, where the agent knows all the possible moves and their effects.
- An **unknown** environment requires the agent to learn the rules through interaction, like playing a new video game without prior instructions.

Importance of These Properties

These properties help in designing agents suitable for specific environments. For example:

- A **fully observable, deterministic, and static** environment allows for simple reflex agents.
- A **partially observable, stochastic, and dynamic** environment requires more complex approaches like model-based or learning agents.

Understanding these characteristics is crucial for selecting the appropriate agent architecture and decision-making strategies.

---

6. The Structure of Agents

The chapter then discusses how agents are structured and how their behavior is implemented. The key idea is that an **agent program** maps percept sequences to actions, and this program runs on some physical **architecture** (like a computer or robot).

- **Table-Driven Agents**: The simplest approach is to use a lookup table that maps every possible percept sequence to an action. However, this is impractical for most real-world problems due to the enormous size of the table.

- **Simple Reflex Agents**: These agents select actions based on the current percept, ignoring the rest of the percept history. They use **condition-action rules** (e.g., "if the car in front is braking, then initiate braking"). However, simple reflex agents are limited because they can’t handle partially observable environments well.

- **Model-Based Reflex Agents**: These agents maintain an **internal state** that depends on the percept history, allowing them to handle partial observability. They use a **model** of the world to update their internal state and choose actions.

- **Goal-Based Agents**: These agents have **goals** that describe desirable situations. They choose actions that will eventually lead to the achievement of their goals. This requires more complex decision-making, often involving **search** and **planning**.

- **Utility-Based Agents**: These agents go beyond goals by using a **utility function** to evaluate how desirable different states are. They choose actions that maximize their expected utility, which allows them to handle trade-offs between conflicting goals.

- **Learning Agents**: These agents can improve their performance over time by learning from their experiences. A learning agent has four components:

    1. **Learning Element**: Responsible for making improvements.
    2. **Performance Element**: Responsible for selecting actions.
    3. **Critic**: Provides feedback on how well the agent is doing.
    4. **Problem Generator**: Suggests exploratory actions to gather new information.

---

7. How Agent Programs Work

Agent programs can use different levels of **environment representation**:

1. **Atomic Representation**: Each state of the world is indivisible (e.g., "the agent is in city A").
2. **Factored Representation**: Each state is split into variables/attributes (e.g., "the agent is in city A with 50% fuel").
3. **Structured Representation**: The world is represented as objects and their relationships (e.g., "the truck is blocking the road").

As the representation becomes more expressive, reasoning and learning become more complex, but the agent can capture more detailed information about the environment.